

A Hierarchical Placement and Short-Hop Routing Framework for Service Function Chain in Space-Air-Ground Integrated Network

Shengpeng Chen, Chunxiao Jiang, *Fellow, IEEE*, Lizhuang Tan, Peiying Zhang

Abstract—Space-air-ground integrated networks (SAGIN) unify satellite, aerial, and terrestrial resources to support mobile edge computing (MEC) with ubiquitous coverage and low end-to-end latency for vehicular intelligence. However, instantiating NFV-based MEC chains over dynamic, heterogeneous SAGIN remains challenging due to the coupling between VNF placement and routing, time-varying links, and large-scale orchestration complexity. We formulate service function chain (SFC) embedding with strict time-to-live (*TTL*) deadlines and measure the quality of edge computation based on the Resource-Latency model. We present SFC-HPR, a two-stage Hierarchical Placement and short hop Routing framework for SAGIN service provisioning. In the placement stage, a deep reinforcement learning agent with parallel graph neural encoders learns shared representations of the substrate network and traffic, yielding low-cost VNF mappings under resource and *TTL* constraints. Leveraging the short hop characteristic of SFC placement, we decompose routing into sub-problems and propose *L-HOP* to solve them exhaustively, thereby maximizing quality of service (QoS). We conduct the experiments on SAGIN topologies of three different scales. Under *TTL*-limit tests, SFC-HPR improves acceptance rate and long-term QoS over the shortest path first (SPF) by 8.40% and 12.95%, respectively. Arrival-rate tests further show acceptance rate gains of 10.57%, 11.15%, and 9.68% over SPF on the three topology scales.

Index Terms—Space-Air-Ground integrated Network, Quality of Service, Latency Constrained Computation, Service Function Chain, Resource Allocation, Software Defined Network, Network Function Virtualization

I. INTRODUCTION

This work is supported by the National Natural Science Foundation of China under Grant 62471493 and 62402257, and partially supported by the Natural Science Foundation of Shandong Province under Grant ZR2023LZH017, ZR2024MF066, ZR2023LZH011, ZR2022LZH015 and 2023QF025, and partially supported by the Taishan Scholar Program under Grant TSQN202312230, and partially supported by the Open Foundation of Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences) under Grant 2023ZD010. (*Corresponding author: Peiying Zhang.*)

Shengpeng Chen and Peiying Zhang are with the Qingdao Institute of Software, College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China, and also with the Shandong Key Laboratory of Intelligent Oil & Gas Industrial Software, Qingdao 266580, China (e-mail: z24070039@s.upc.edu.cn; zhangpeiying@upc.edu.cn).

Chunxiao Jiang is with the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: jchx@tsinghua.edu.cn).

Lizhuang Tan and Peiying Zhang are with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China and also with the Shandong Provincial Key Laboratory of Computing Power Internet and Service Computing, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China (e-mail: tanlzh@sdsas.org).

By integrating space, air, and ground resources, the Space-Air-Ground Integrated Network (SAGIN) emerges as a critical architecture for ensuring wide-area coverage, low latency, and high reliability [1], [2]. SAGIN unifies satellite constellations, unmanned aerial vehicles (UAVs), high-altitude platforms (HAPs), and terrestrial base stations within a collaborative framework. Through layered coverage and distributed computing capabilities across these platforms, SAGIN enables seamless connectivity and rapid service deployment [3], [4], [5], [6]. Simultaneously, Mobile Edge Computing (MEC) pushes computing and caching resources down to satellites, HAPs, and ground stations. Bringing resources closer to users significantly reduces backhaul bandwidth consumption and accelerates response speeds for delay-sensitive applications.

The introduction of Network Function Virtualization (NFV) and Service Function Chaining (SFC) [7] enables on-demand service orchestration. By linking virtual network functions (VNFs) such as firewalls and traffic monitors across heterogeneous infrastructure, these technologies enforce data flows to traverse inter-satellite links (ISLs), aerial relays, and terrestrial backhaul networks in specific sequences. Fig. 1 illustrates the workflow for Task A, involving precise vehicle positioning, where raw measurements are first cleaned and fused by three aerial MEC nodes before being corrected and distributed by a satellite. This process minimizes costs while satisfying end-to-end (E2E) latency requirements [8]. Task B demonstrates an encrypted data synchronization scheme [9]. After the sending vehicle completes session key negotiation, traffic is forwarded to the destination while auxiliary VNFs are anchored at trusted gateway edge nodes to simplify processing over constrained links without altering encryption semantics.

However, instantiating SFCs in highly dynamic and heterogeneous SAGIN environments faces severe cross-domain coordination challenges. SAGIN is characterized by high three-dimensional topological dynamics, intermittent ISL connectivity, substantial cross-domain propagation delays, and extremely unbalanced computing and energy resources across platforms. Neglecting these issues during SFC deployment often leads to VNFs being scattered across distant cross-domain nodes, resulting in severe network detours, latency violations, and resource waste. Consequently, placement and routing exhibit strong coupling in SAGIN. This joint optimization of resource-constrained SFC placement and routing (SFCPR) is a strong NP-hard problem, rendering exact algorithms computationally infeasible for large-scale or online requests [10]. Although Software Defined Networking (SDN) and NFV orchestrators facilitate cross-layer coordination [11], [12], existing research remains inadequate in addressing

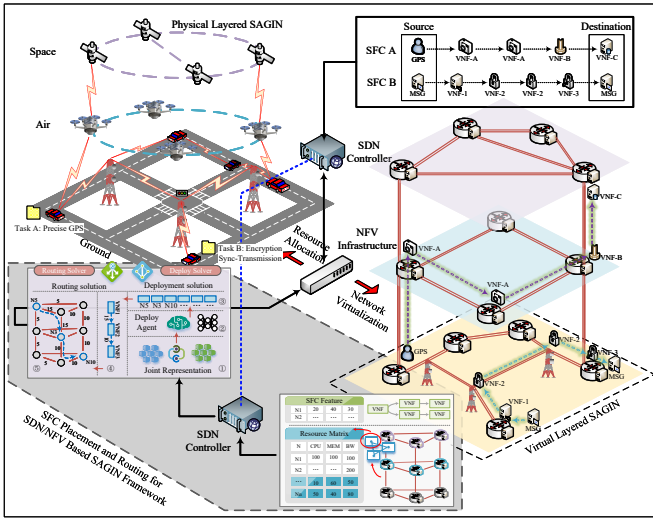


Fig. 1. SFC placement and routing problems in SDN/NFV-enabled SAGIN.

complex MEC constraints and real-time joint optimization. Most methods either decouple placement from routing or introduce prohibitive computational overhead [13], [14].

To address these limitations, this paper investigates the complex SFC mapping problem under strict latency constraints. In delay-sensitive networks like SAGIN, service tasks possess rigorous time-to-live (TTL) thresholds. Notably, under strict delay constraints and high cross-domain communication costs, but in the absence of task-specific constraints, optimal SFC mapping in SAGIN typically exhibits significant *local clustering*. Physical links in the underlying infrastructure have capacity upper bounds. To ensure the aggregate traffic of all virtual links mapped to a physical link does not exceed its maximum capacity, mapping VNFs of the same SFC to geographically proximate computing nodes reduces the consumption of expensive link bandwidth. An extreme solution involves co-locating all VNFs on the same rack or server, provided physical resources are sufficient, to avoid bandwidth usage for inter-VNF data exchange. However, such deployment leads to instantaneous failure of the entire SFC if the underlying equipment goes offline, necessitating physical isolation through mutually exclusive VNF placement to avoid single points of failure (SPOF). Although it is impractical for VNFs to be co-located on a single node, deploying them on different topologically close nodes to maintain *local clustering* remains highly beneficial for SAGIN. Reducing hop counts effectively lowers propagation delay, which is typically much larger in SAGIN links than in conventional terrestrial networks [15]. Consequently, the short-hop characteristic inherently bounds the search space for routing optimization between adjacent VNFs, rendering its computational complexity highly manageable. Motivated by this, we propose a two-stage SFC provisioning strategy. Initially, a deep reinforcement learning (DRL) model is deployed to generate placement schemes characterized by *local clustering*. Subsequently, we introduce a routing method explicitly tailored to align with service objectives. Because the clustered placement naturally curtails routing complexity, this method is straightforward to implement and avoids computational inflation, thereby achieving quality of service (QoS) enhancements without incurring additional overhead. The primary contributions are

summarized as follows:

- To efficiently solve the SFCPR problem with latency constraints, we propose a two-stage hierarchical placement-routing framework named SFC-HPR. In the first stage, we design a DRL placement agent. We utilize parallel Graph Convolutional Network (GCN) encoders to extract features from the SAGIN physical network and SFC requests to form shared representations. Under an Actor-Critic architecture, the decoder agent is trained using complete SFC trajectories to stabilize policy updates and generate reliable placement actions.
- we develop a bounded-search routing mechanism that exploits the short-hop structure induced by the placement stage. The mechanism performs QoS-aware exhaustive selection within an L -hop feasible set, explicitly accounts for bandwidth and remaining latency constraints, and uses a fallback shortest path procedure for cases outside the bounded search region. This placement-routing co-design provides a controllable balance between routing quality and computational cost.
- Finally, we construct a hierarchical SAGIN simulation environment. Extensive comparative experiments demonstrate that, compared to existing two-stage heuristics and reinforcement learning baselines, SFC-HPR significantly improves the SFC acceptance rate without increasing latency overhead and achieves substantial advantages in computing resource QoS.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system model and problem formulation. Section IV details the proposed SFC-HPR framework. Section V reports extensive experimental validation and performance analysis. Section VI concludes the paper and outlines future research directions.

II. RELATED WORK

Existing research on service function chain embedding in SAGIN can be broadly categorized into three lines: VNF placement (SFCP), virtual link routing (SFCR), and joint placement and routing (SFCPR).

A. SFCP: SFC Placement in SAGIN

The SFCP problem in SAGIN concerns the selection of physical nodes for hosting the VNFs of an SFC under constraints on computation, bandwidth, and latency. It is commonly formulated as an integer linear or nonlinear programming problem and is a classical NP-hard combinatorial optimization problem [16], [13]. Exact methods are generally limited to small-scale instances because the computational complexity grows exponentially with network size, which restricts their applicability in large-scale SAGIN environments. As a result, scalable heuristic and metaheuristic algorithms have been widely adopted. For example, Wang et al. [17] proposed a feature-weighted greedy algorithm for SFC planning in SAGIN, whereas Li et al. [11] developed a tabu-search-based method for joint VNF mapping and rescheduling in dynamic scenarios. Although these heuristic methods are computationally efficient, they often converge to local optima. By contrast, DRL is well suited to dynamic environments

because it supports online learning and adaptation to topological changes [12]. Jia et al. [13] introduced a reconstructed time-slot graph to partition the time domain, modeled the topology as quasi-static within each slot, and formulated SFC scheduling as a Markov Decision Process (MDP) solved by DRL. Despite this adaptability, DRL requires complex training procedures and typically does not provide optimality guarantees. To address the dynamics of SAGIN, hierarchical collaborative control enables unified management of satellite, aerial, and ground devices through multilevel distributed SDN [12]. In addition, SDN/NFV technologies provide the foundation for resource orchestration in SAGIN and support centralized or hierarchical scheduling architectures [11].

B. SFCR: Virtual Link Routing in SAGIN

Traditional approaches often formulate SFCR as a multi-commodity flow problem or as a latency-constrained shortest-path problem [18], [19]. In general, existing studies either restrict routing to a feasible shortest path or identify candidate paths by using multi-source shortest-path algorithms and then jointly optimize routing and placement [20]. Although studies that specifically address SFCR in SAGIN are still limited, path-mapping techniques developed for Virtual Network Embedding (VNE) remain applicable [21], [22].

C. Joint SFCPR: Joint Placement and Routing in SAGIN

The joint SFCPR problem simultaneously determines VNF placement and routing, and therefore constitutes a complete embedding task [23], [24]. This strongly NP-hard problem includes classical challenges such as facility location and multi-commodity flow [25], [26]. Unlike decoupled optimization strategies, joint approaches globally balance computational and transport resources, but this advantage comes at the cost of higher dimensionality and greater computational complexity [25]. Under strict upper bounds on the E2E latency of SFC, the problem becomes LC-SFCPR, which is a more challenging variant that requires the simultaneous satisfaction of stringent delay constraints. Existing studies on TTL constraints [27], [28] typically employ layered-graph constructions together with enhanced search or shortest-path algorithms. Although most of these studies focus on terrestrial or edge scenarios, they offer useful insights for SAGIN settings with strict latency requirements. Nevertheless, TTL -constrained SFC embedding that is tailored to the multilayer collaboration of SAGIN still requires further investigation.

III. SYSTEM MODEL AND PROBLEM FORMULATION

This section elaborates on the system model and provides a detailed formulation of the problem. A summary of the primary notations used throughout this paper, along with their definitions, is presented in Table I.

A. System Model

1) *SAGIN Topology*: Similar to study [29], we model the SAGIN architecture as a weighted undirected layered graph $G^{pn} = (V^{pn}, E^{pn})$, where V^{pn} and E^{pn} denote the sets of physical vertices and physical links, respectively. According to their spatial domains, the physical links are categorized

TABLE I
SUMMARY OF MAJOR NOTATIONS AND DEFINITIONS.

System Model	
V^{pn}, E^{pn}	Set of vertexes and edges within the physical network
V^{sfc}, E^{sfc}	Set of vertexes and edges defined by the SFC
$R_{c,m}^{pn}(v), R_{bw}^{pn}(e)$	Physical resource capacities
$\hat{R}_{c,m}^{pn}(v), \hat{R}_{bw}^{pn}(e)$	Residual physical resources
$R_{c,m}^{sfc}(v), R_{bw}^{sfc}(e)$	Resource demands of SFC
t_p	Processing delay at a physical vertex
t_d	Propagation delay along a physical edge
TTL^{sfc}	Strict Time-to-Live constraint of the SFC
Problem Formulation and Definitions	
π^p, π^r	Placement and routing policies
$f^{\pi^p}(v^{sfc}) \rightarrow v^{pn}$	Mapping of a virtual vertex to a physical vertex under placement policy π^p
$f^{\pi^r}(e^{sfc}) \rightarrow P^{pn}$	Mapping of a virtual edge to a physical path under routing policy π^r
P^{pn}	A simple path within the physical network
$t^{P^{pn}}$	Delay associated with path P^{pn}
T^{sfc}	Total latency of the SFC
Ψ^{sfc}	Expected revenue generated by the SFC
Q^{sfc}	Overall resource QoS metric for the SFC
$\bar{Q}$	Long-term resource QoS of the system
G^V, G^E	Success indicators for the SFCP and SFCR problems, respectively
$AccR, AccN$	Acceptance rate and the total number of accepted SFCs
$\bar{LaT}$	Average latency of the SFCs
J^p, J^r	Objective functions for placement and routing

into six types: space to space (S2S), space to air (S2A), space to ground (S2G), air to air (A2A), air to ground (A2G), and ground to ground (G2G). Here, ground nodes include both vehicles and base stations (BS); thus, G2G links cover vehicle to vehicle (V2V), vehicle to base station (V2B), and base station to base station (B2B) connections. Assuming uniform CPU clock speeds, we quantify computational performance based on the core count. Each vertex is characterized by its number of CPU cores, memory capacity, and processing latency, denoted as $v_i^{pn} = [R_c^{pn}, R_m^{pn}, t_p]$. Each edge $e_i^{pn} \in E^{pn}$ is defined as a tuple $e_i^{pn} = [v_{src}^{pn}, v_{dst}^{pn}, R_{bw}^{pn}, t_d]$, comprising the source node, destination node, maximum bandwidth, and propagation latency.

2) *SFC Request*: We address the SFCPR problem subject to strict latency constraints by modeling each SFC as a directed weighted graph $\bar{G}^{sfc} = \{V^{sfc} \cup E^{sfc}, TTL^{sfc}\}$. Each SFC request is a deadline-constrained task with a maximum residence time. In this work, TTL^{sfc} denotes the lifetime of an SFC request. The request must complete its E2E processing and transmission before its TTL expires, and therefore its latency must satisfy $T^{sfc} \leq TTL^{sfc}$. Once the request is admitted, the allocated computing and bandwidth resources remain reserved for TTL^{sfc} time units and are released when the request expires. An SFC consists of an ordered sequence of VNFs. Although different chains may share underlying resources, VNFs belonging to the same chain must be mapped to distinct physical vertexes. The resource requirements of a VNF are modeled as $v^{sfc} = [R_c^{sfc}, R_m^{sfc}]$ (representing CPU and memory demands, respectively). Each virtual edge is defined by the triplet $e_i^{sfc} = [v_{src}^{sfc}, v_{dst}^{sfc}, R_{bw}^{sfc}]$, which specifies the source, destination, and bandwidth requirement.

B. Problem Definition

SFC embedding is typically decomposed into two independent sub-problems: vertex placement and edge mapping. As illustrated in Fig. 1, the placement policy first determines the selection of vertexes; once the node positions are established, the routing policy maps each virtual edge to a physical path.

1) *Two-Stage Policy for SFC Placement and Routing*: The SFCP problem is defined as:

$$\text{SFCP} : V^{sfc} \rightarrow V^{pn}, \text{ for } \forall v_i^{sfc} \in V^{sfc}. \quad (1)$$

The first stage requires the mapping $V^{sfc} \rightarrow V^{pn}$ to be fully satisfied, identifying **at least** one subset of vertexes via the mapping function f^{π^p} . Here, f^{π^p} maps $\{v_1^{sfc}, \dots, v_n^{sfc}\}$ to V^{pn} , denoted as $f^{\pi^p}(v_i^{sfc}) \rightarrow v_j^{pn}$. The constraints are defined as follows:

$$C1 : R_c^{sfc}(v_i^{sfc}) \leq \hat{R}_c^{pn}(f^{\pi^p}(v_i^{sfc})), \text{ for } \forall v_i^{sfc} \in V^{sfc}, \quad (2)$$

$$C2 : R_m^{sfc}(v_i^{sfc}) \leq \hat{R}_m^{pn}(f^{\pi^p}(v_i^{sfc})), \text{ for } \forall v_i^{sfc} \in V^{sfc}, \quad (3)$$

where $\hat{R}_*^{pn}$ denotes the residual physical resources. Successful placement necessitates the simultaneous satisfaction of $C1$ and $C2$.

The SFCR is formulated as:

$$\text{SFCR} : E^{sfc} \rightarrow E^{pn}, \text{ for } \forall e_i^{sfc} \in E^{sfc}. \quad (4)$$

This operation maps the virtual network to physical paths via the function $f^{\pi^r}(e_i^{sfc}) \rightarrow P_j^{pn} \subseteq G^{pn}$. A path $P^{pn} = \{V_P^{pn} \cup E_P^{pn}\} \subseteq G^{pn}$ comprises m continuous edges, where $V_P^{pn} = \{v_1^{pn}, \dots, v_{m+1}^{pn}\}$ and $E_P^{pn} = \{e_1^{pn}, \dots, e_m^{pn}\}$. For convenience, we focus primarily on the edge set and refine the f^{π^r} as $f^{\pi^r}(e_i^{sfc}) \rightarrow E_{P_i}^{pn}$, meaning that the virtual edge e_i^{sfc} is mapped to the physical link P_i , whose edge set is $E_{P_i}^{pn}$. The selected path must satisfy constraint $C3$:

$$C3 : R_{bw}^{sfc}(e_i^{sfc}) \leq \min_{e_j^{pn} \in f^{\pi^r}(e_i^{sfc})} \hat{R}_{bw}^{pn}(e_j^{pn}). \quad (5)$$

Finding a solution for $E^{sfc} \rightarrow E^{pn}$ requires identifying at least one satisfying subset of edges. It is important to note that multiple valid solutions may exist for a single E^{sfc} .

A request sfc is accepted if both $\mathcal{G}^{V^{sfc}}$ and $\mathcal{G}^{E^{sfc}}$ are successful, thereby occupying physical resources when constraints $C1$ through $C4$ are met. The residual resources in Eq.(2, 3, 5) are updated as follows:

$$\begin{aligned} \hat{R}_*^{pn} \leftarrow & \begin{cases} \hat{R}_c^{pn}(v_i^{pn}) - R_c^{sfc}(v_j^{sfc}), \\ \hat{R}_m^{pn}(v_i^{pn}) - R_m^{sfc}(v_j^{sfc}), \\ \hat{R}_{bw}^{pn}(e_i^{pn}) - R_{bw}^{sfc}(e_j^{sfc}), \end{cases} \\ & \text{for } \forall (v_j^{sfc}, v_i^{pn}) \in V^{sfc} \times f^{\pi^p}(v_j^{sfc}), \\ & \text{for } \forall (e_j^{sfc}, e_i^{pn}) \in E^{sfc} \times f^{\pi^r}(e_j^{sfc}), \end{aligned} \quad (6)$$

where $P_j^{pn} = f^{\pi^r}(e_j^{sfc})$ and $v_i^{pn} = f^{\pi^p}(v_j^{sfc})$.

The latency of a physical path P_i^{pn} is calculated as $T_{P_i^{pn}} = \sum_{v_j^{pn} \in P_i^{pn}} t_p^{v_j^{pn}} + \sum_{e_j^{pn} \in P_i^{pn}} t_d^{e_j^{pn}}$. The total latency of an SFC, denoted by T^{sfc} , is defined as the sum of the processing delays of

the mapped physical vertices and the propagation delays of the mapped physical edges:

$$\begin{aligned} T^{sfc} &= t_p^{V^{sfc} \rightarrow V^{pn}} + t_d^{E^{sfc} \rightarrow E^{pn}}, \\ &= \sum_{v_i^{sfc}} t_p^{f^{\pi^p}(v_i^{sfc})} + \sum_{e_i^{sfc}} \sum_{e_j^{pn} \in f^{\pi^r}(e_i^{sfc})} t_d^{e_j^{pn}}, \end{aligned} \quad (7)$$

To ensure service viability, the delay of the SFC must satisfy constraint $C4$. If this constraint is violated, the placement task and routing task SFCR defined in (1, 4) is considered a failure:

$$C4 : T^{sfc} \leq TTL^{sfc}. \quad (8)$$

Based on the definitions above, we formulate the LC-SFCPR problem as:

$$\begin{aligned} \text{LC-SFCPR} : \vec{G}^{sfc} \rightarrow G^{pn}, \text{ for } \forall sfc \in sfc_r, \\ \text{s.t. } C1, C2, C3, C4, \end{aligned} \quad (9)$$

where sfc_r denotes the temporally ordered set of requests. We treat $\mathcal{G}$ as a binary indicator of embedding success:

$$\begin{cases} \mathcal{G}^{V^{sfc}} = 1, & \text{if } V^{sfc} \rightarrow V^{pn} \text{ succeeds,} \\ \mathcal{G}^{V^{sfc}} = 0, & \text{if } V^{sfc} \rightarrow V^{pn} \text{ fails,} \\ \mathcal{G}^{E^{sfc}} = 1, & \text{if } E^{sfc} \rightarrow E^{pn} \text{ succeeds,} \\ \mathcal{G}^{E^{sfc}} = 0, & \text{if } E^{sfc} \rightarrow E^{pn} \text{ fails.} \end{cases} \quad (10)$$

C. Resource-Latency based Quality of Service Model

We propose a *Resource-Latency* model to evaluate the quality of service, taking into account both computing and transmission resources. The embedding revenue of an SFC is defined as the aggregate of the required resources:

$$\Psi^{sfc} = \sum_{v_i^{sfc}} \left(R_c^{sfc}(v_i^{sfc}) + R_m^{sfc}(v_i^{sfc}) \right) + \sum_{e_i^{sfc}} R_{bw}^{sfc}(e_i^{sfc}). \quad (11)$$

Let $B_{\text{req}}^{sfc} = \sum_{e_i^{sfc}} R_{bw}^{sfc}(e_i^{sfc})$ be the total bandwidth requested by the virtual links, and $B_{\text{alloc}}^{sfc} = \sum_{e_i^{sfc}} h_i R_{bw}^{sfc}(e_i^{sfc})$ be the aggregate physical-link bandwidth occupied by the embedding, where $h_i = |E_P^{pn} \cap f^{\pi^r}(e_i^{sfc})|$ counts the physical links in the path assigned to virtual link e_i^{sfc} . To quantify latency-aware resource efficiency, we introduce a metric Q^{sfc} , defined as follows:

$$Q^{sfc} = \frac{\Psi^{sfc}}{T^{sfc}} \eta_{bw}^{sfc}, \quad \eta_{bw}^{sfc} = \frac{B_{\text{req}}^{sfc}}{B_{\text{alloc}}^{sfc}}, \quad (12)$$

where $\frac{\Psi^{sfc}}{T^{sfc}}$ denotes the Resource-Latency ratio. For the same request, lower latency therefore yields a higher Resource-Latency value. The second term is a bandwidth efficiency factor, not a conventional link-utilization ratio. It can be expressed as $\eta_{bw}^{sfc} = \frac{\sum_{e_i^{sfc}} R_{bw}^{sfc}(e_i^{sfc})}{\sum_{e_i^{sfc}} |E_P^{pn} \cap f^{\pi^r}(e_i^{sfc})| \times R_{bw}^{sfc}(e_i^{sfc})} \leq 1$,

the inverse of the bandwidth-weighted average number of physical links carrying each virtual link. The factor reaches one when every virtual link maps to a single physical link and decreases when the same bandwidth demand is reserved over more physical links. Thus, for a fixed SFC request, maximizing Q^{sfc} is equivalent to minimizing the product

$T^{sfc} B_{\text{alloc}}^{sfc}$. Eq. (12) consequently favors embeddings that provide the requested service with both low E2E latency and a small aggregate bandwidth footprint. The factor η_{bw}^{sfc} indirectly limits one fragmentation source: multi-hop mapping spreads bandwidth reservation over more physical links and can reduce subsequent-request feasibility.

D. Performance metrics

The long-term acceptance rate, denoted as $AccR$, is defined as the ratio of accepted SFCs to the total number of requests:

$$\begin{aligned} AccR &= \frac{1}{|sfc|} AccN, \\ &= \frac{1}{|sfc|} \sum_{sfc} (\mathcal{G}^{V^{sfc}} \mathcal{G}^{E^{sfc}}), \end{aligned} \quad (13)$$

where $AccN$ represents the total count of accepted SFCs, and $|sfc|$ denotes the total number of requests.

The overall average latency of the embedded SFCs, denoted by $\overline{LaT}$, is calculated as:

$$\overline{LaT} = \frac{1}{AccN} \sum_{sfc} (\mathcal{G}^{V^{sfc}} \mathcal{G}^{E^{sfc}} T^{sfc}). \quad (14)$$

To maximize the long-term cumulative service quality, we define the average QoS metric $\overline{Q}$ as:

$$\overline{Q} = \frac{1}{T^{total}} \sum_{sfc} (\mathcal{G}^{V^{sfc}} \mathcal{G}^{E^{sfc}} Q^{sfc}), \quad (15)$$

where T^{total} indicates the total operational time of the system.

E. Optimization Objective

The SFCPR objective encompasses two aspects. First, we identify an optimal candidate vertex strategy to maximize gains. Second, the routing phase maximizes $AccR$ and $\overline{Q}$ without compromising latency LaT , formulated as:

$$\begin{aligned} J &= \{J^p, J^r\}, \\ \text{s.t. } J^p &= \mathbf{maximize} \{AccR, -LaT, \overline{Q}\}, \\ J^r &= \mathbf{maximize} \{AccR, \overline{Q}\} | J^p. \end{aligned} \quad (16)$$

IV. HIERARCHICAL PLACEMENT AND ROUTING OPTIMIZATION ALGORITHM

In this section, we present the proposed SFC-HPR algorithm and detail the Markov Decision Process (MDP) formulation for the LC-SFCPR problem. The system is decomposed into two distinct sub-problems: SFCP and SFCR. These are solved by specific sub-models, which are denoted as the **placement policy** π^p and the **routing policy** π^r .

A. DRL-based VNF Placement

The policy π^p seeks to identify a placement function f^{π^p} that satisfies the constraints $C1$ and $C2$ established in Eq (2, 3). Let $sfc_{k,t}$ denote the k^{th} SFC request at time t ; for brevity, the subscripts k and t remain omitted in the subsequent discussion. For each sfc , π^p takes the virtual vertex set V^{sfc} as input and outputs a subset of physical vertexes $\bar{V}^{pn} \subseteq V^{pn}$ of equal cardinality, as defined in Eq (1). As illustrated in Fig. 2, the

network status (comprising vertex status, residual computing resources, link delay, and bandwidth) is essential for decision-making. We construct a feature matrix $M \in \mathbb{R}^{|V^{pn}| \times f}$ to represent this status. Each physical vertex is characterized by a resource vector $\vec{f}_i \in \mathbb{R}^f$, and the overall state is defined as follows:

$$M = [\vec{f}_1 \ \vec{f}_2 \ \cdots \ \vec{f}_{|V^{pn}|}]^T. \quad (17)$$

The vector $\vec{f}_i$ encapsulates the attributes of vertex v_i^{pn} , including residual resources (CPU, memory, average adjacency bandwidth) and the node degree. Specifically, this is defined as $\vec{f}_i = [\hat{R}_c^{pn}, \hat{R}_m^{pn}, \hat{R}_{bw}^{pn}, dg]$.

Following the approach in [30], the encoder network is composed of multiple GCNs [31]. These networks generate embedding vectors for both physical and virtual vertexes. We design each GCN block to perform matrix multiplication followed by a linear layer. The output of the encoder is given by:

$$M^{(h+1)} = \mathbf{Leaky\ ReLU}(\hat{A}M^{(h)}W^{(h)}), \quad (18)$$

where $\hat{A}$ represents the normalized adjacency matrix (for either G^{pn} or $\bar{G}^{sfc}$) and $W^{(h)}$ denotes the learnable weights of block h . Two distinct encoder networks process the virtual and physical networks to produce the embeddings $\mathcal{E}^{pn} \in \mathbb{R}^{|V^{pn}| \times H}$ and $\mathcal{E}^{sfc} \in \mathbb{R}^{|V^{sfc}| \times H}$. The **embedding state representation** S for a given sfc concatenates the current network state with these embeddings:

$$\begin{aligned} S^{sfc} &= \{s_1, \dots, s_i, \dots, s_{|V^{sfc}|}\}, \\ \text{s.t. } s_i &= [\mathcal{E}^{pn} \parallel \chi \parallel \mathcal{D}_{v_{pre}^{sfc}} \parallel \mathcal{E}_i^{sfc}], \end{aligned} \quad (19)$$

where $s_i \in \mathbb{R}^{2H+2|V^{pn}|}$ represents temporally continuous states and $\mathcal{E}_i^{sfc}$ denotes the vector corresponding to the i -th VNF. The term $\mathcal{D}_{v_{pre}^{sfc}} \in \mathbb{R}^{|V^{pn}|}$ denotes the distances from the physical vertexes to the location of the previous action. The binary mask $\chi \in \mathbb{R}^{|V^{pn}|}$ identifies unselected vertexes that possess sufficient resources (set to 1 if valid, and 0 otherwise).

Using these representations, a three-layer Multi-Layer Perceptron (MLP) decision layer converts the state into a probability distribution over $\mathbb{R}^{|V^{pn}|}$. Based on this distribution, the **action** involves selecting a physical vertex v_i^{pn} from the candidate set V^{pn} to host the virtual vertex v_i^{sfc} :

$$\begin{aligned} A^{sfc} &= \{a_1, \dots, a_i, \dots, a_{|V^{sfc}|}\}, \\ \text{s.t. } a_i &\sim \mathcal{P} = \mathbf{softmax}_{\chi}(MLP(s_i)). \end{aligned} \quad (20)$$

The **reward function** is designed to guide the Reinforcement Learning (RL) model toward maximizing long-term rewards. Regarding the objective J^p in Eq (16), the metrics Q^{sfc} and $AccR$ are positively correlated with $\overline{Q}$. Since $AccR$ is difficult to represent numerically within the immediate reward structure, we adopt the instantaneous QoS revenue Q_i^{sfc} of action a_i as the reward metric:

$$\begin{aligned} R^{sfc} &= \{r_1, \dots, r_i, \dots, r_{|V^{sfc}|}\}, \\ \text{s.t. } r_i &= \begin{cases} Q_i^{sfc} & \text{if } sfc \text{ is successful,} \\ -\alpha & \text{otherwise,} \end{cases} \\ Q_i^{sfc} &= \left[\frac{R_c^{sfc}(v_i^{sfc}) + R_m^{sfc}(v_i^{sfc})}{\sum_{e_i^{sfc} \in V^{sfc}} R_{bw}^{sfc}(e_i^{sfc})} + \right] \frac{R_{bw}^{sfc}}{R_{bw}^{sfc \rightarrow pn}}. \end{aligned} \quad (21)$$

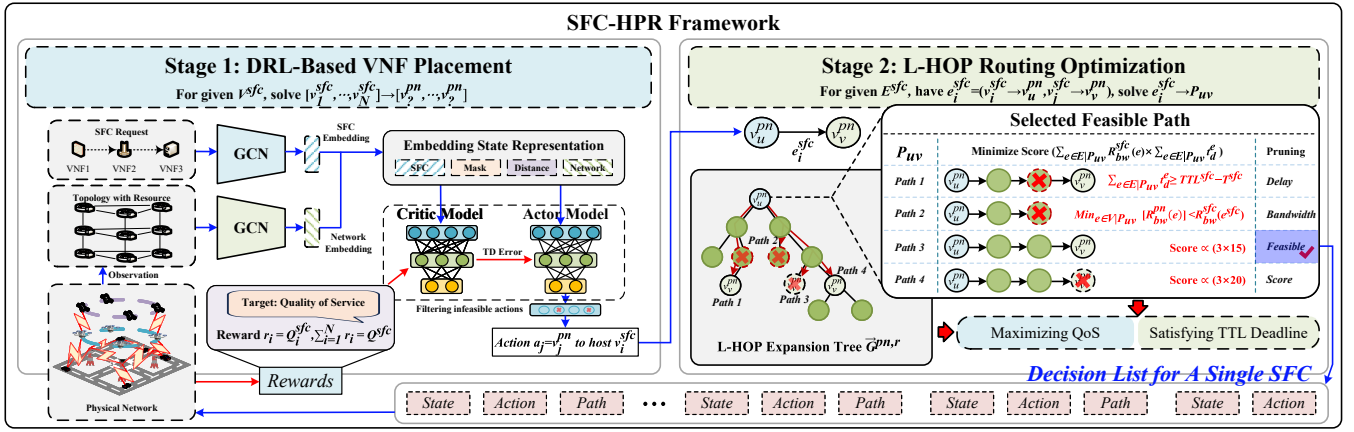


Fig. 2. The structure of the SFC-HPR Framework With DRL-Based VNF Placement and L -HOP Routing Optimization.

As shown in Fig. 2, the post-routing reward in Eq. (21) is unavailable immediately after an individual placement action. For an SFC request, the placement policy first generates the complete ordered trajectory $\tau^{sfc} = (s_1, a_1), \dots, (s_{|V^{sfc}|}, a_{|V^{sfc}|})$. During this process, each placement step's state, action, policy log-probability, and value estimate are stored while its final reward remains pending. After all VNFs are placed, the routing stage maps the virtual links between consecutive VNFs. Only then can the embedding-success indicator, E2E latency T^{sfc} , and bandwidth efficiency η_{bw}^{sfc} be determined.

The resulting SFC-level outcome is assigned to every stored placement step. Upon successful embedding, the i -th action receives $r_i = Q_i^{sfc}$. This construction satisfies $\sum_{i=1}^{|V^{sfc}|} r_i = Q^{sfc}$. If either placement or routing fails, all actions in that SFC trajectory receive the penalty $-\alpha$. After assigning the reward vector, the returns and advantages are computed over the completed trajectory, and the actor, critic, and shared encoder are updated. No policy update uses an incomplete SFC trajectory. This delayed assignment is adopted because an early placement action changes the feasible nodes available to later VNFs and determines the physical endpoints of subsequent routing tasks. Its quality therefore cannot be evaluated solely from the locally allocated computing resources. The post-routing reward incorporates the eventual E2E latency and physical-link bandwidth footprint into the learning signal for every placement step.

B. L -HOP Routing Optimization

The central design principle of L -HOP is to exploit the structural coupling between VNF placement and virtual-link routing. The TTL constraint in Eq. (8), the bandwidth-utilization term in Eq. (12), and the joint objective in Eq. (16) jointly encourage consecutive VNFs to be placed within a small topological neighborhood, without imposing a hard bound on their physical distance. As a result, the placement stage tends to transform the original network-wide routing problem into a set of localized routing subtasks:

$$P = \{P_1, \dots, P_i, \dots, P_{|E^{sfc}|}\}, \quad (22)$$

$$\text{s.t. } P_i = \pi^r(e_i^{sfc}).$$

For each virtual link $e_i^{sfc} = (v_u^{sfc}, v_v^{sfc})$, the placement policy π^p determines the corresponding physical endpoints $v_u^{pn} = f^{\pi^p}(v_u^{sfc})$ and $v_v^{pn} = f^{\pi^p}(v_v^{sfc})$.

Instead of explicitly constructing the entire L -HOP expansion tree, three feasibility checks are applied online: 1) *hop pruning*, which terminates any branch whose depth exceeds L ; 2) *bandwidth pruning*, which discards an edge e^{pn} if its residual bandwidth $\hat{R}_{bw}^{pn}(e^{pn})$ is smaller than the demand $R_{bw}^{sfc}(e_i^{sfc})$; and 3) *delay pruning*, which backtracks immediately once the accumulated delay exceeds $\hat{T}^{sfc}$. In addition, after the first feasible path is found, its score is used as an incumbent upper bound, and any branch whose lower-bound score is no better than the incumbent is further pruned. Among all feasible candidates, the algorithm returns the path with the minimum score:

$$\text{score}(P_i) = \left(\sum_{e^{pn} \in P_i} R_{bw}^{sfc}(e_i^{sfc}) \right) \left(\sum_{e^{pn} \in P_i} t_d^{e^{pn}} \right) \quad (23)$$

$$= (|P_i| - 1) R_{bw}^{sfc}(e_i^{sfc}) \sum_{e^{pn} \in P_i} t_d^{e^{pn}}.$$

Eq. (23) is qualitatively aligned with the Resource-Latency objective in Eq. (12). The stage two presented in Fig. 2 illustrates an example with $L = 3$: starting from u , the DFS recursively explores feasible outgoing substrate links, updates the cumulative state, and backtracks as soon as any pruning condition is triggered. The final output is the feasible path from u to v with the smallest score. If no feasible path is found within the prescribed L hops, the system falls back to the Dijkstra algorithm to preserve robustness.

Given fixed physical endpoints and a prescribed hop bound L , the pruning procedure enumerates all feasible simple paths with at most L hops and selects the path that minimizes the score in Eq. (23). Its optimality is therefore limited to this bounded feasible candidate set. This behavior differs from SPF, which commits to one path under a shortest-path metric and may miss an alternative path with a better resource-delay trade-off. The bounded enumeration also avoids the rapidly growing search space induced by unrestricted path enumeration. Thus, L controls the trade-off between candidate-path coverage and routing cost, while the Dijkstra fallback maintains feasibility when no candidate path exists within the bounded neighborhood.

C. Training Framework

We employ the Advantage Actor-Critic (A2C) algorithm as the training framework. As indicated in Eq. (20), we

construct two independent MLPs to serve as the actor and critic networks, respectively. These networks perform policy learning based on the shared encoder embedding defined in Eq. (18). In accordance with the MDP formulation presented in Section IV-A, the proposed algorithm operates at the granularity of the SFC, in contrast to the isolated VNF-level settings frequently utilized in prior studies. This approach ensures a comprehensive evaluation of the topologically ordered action sequence within an SFC, where the decision at each step determines the context for subsequent actions. The deployment of an SFC is considered successful if and only if the placement and routing strategies identify feasible solutions for their respective sub-problems (specifically, the sets of candidate vertices and paths). Based on the reward function defined in Eq. (21), the objective of the DRL agent is to maximize the expected long-term return:

$$J(\theta_a) = \mathbb{E}_{\pi_{\theta_a}^p} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right], \quad (24)$$

where $\pi_{\theta_a}^p$ denotes the actor policy network with learnable parameters θ_a , and $\gamma \in (0, 1]$ represents the discount factor.

By defining the state-value function $V_{\theta_c}(s)$ and computing the critic loss as the Mean Squared Error (MSE) relative to the Temporal Difference (TD) target, we optimize the shared encoder by combining the policy and value losses as follows:

$$\begin{aligned} \mathcal{L}(\theta_a, \theta_c, \theta_e) = & -\mathbb{E}_t \left[\underbrace{\left[\log \pi_{\theta_a}^p(a_t | s_t, \theta_e) \cdot \hat{A}(s_t, a_t) \right]}_{\text{Policy Gradient Term}} \right. \\ & + \underbrace{\left[\frac{\beta}{\log |V^{pn}|} \mathcal{H}(\pi_{\theta_a}^p(\cdot | s_t, \theta_e)) \right]}_{\text{Entropy Bonus}} \left. \right] \\ & + \underbrace{c_v \mathbb{E}_t[(\hat{R}_t - V_{\theta_c}(s_t, \theta_e))^2]}_{\text{Critic Loss}}, \quad (25) \\ \text{s.t. } & \hat{R}_t = r_t + \gamma V_{\theta_c}(s_{t+1}, \theta_e), \\ & \hat{A}(s_t, a_t) = \hat{R}_t - V_{\theta_c}(s_t, \theta_e), \\ & \mathcal{H}(\pi(\cdot | s)) = - \sum_a \pi(a | s) \log \pi(a | s), \end{aligned}$$

where $\hat{A}(s_t, a_t)$ is the estimated advantage for the return $\hat{R}_t$, c_v serves as a weighting factor for the value loss, and $\frac{\beta}{\log |V^{pn}|}$ acts as a regularized entropy coefficient that balances exploration and exploitation. Here, s_t, θ_e represents the state embedding generated by the shared encoder parameterized by θ_e . The parameters are updated via gradient descent:

$$\begin{aligned} \theta_a & \leftarrow \theta_a - \alpha_a \nabla_{\theta_a} \mathcal{L}(\theta_a, \theta_c, \theta_e), \\ \theta_c & \leftarrow \theta_c - \alpha_c \nabla_{\theta_c} \mathcal{L}(\theta_a, \theta_c, \theta_e), \\ \theta_e & \leftarrow \theta_e - \alpha_e \nabla_{\theta_e} \mathcal{L}(\theta_a, \theta_c, \theta_e), \end{aligned} \quad (26)$$

where α_a , α_c , and α_e denote the learning rates for the actor, critic, and encoder, respectively.

D. Complexity Analysis

1) *Routing policy*: For a single sub-routing task P_i , the DFS explores only simple paths within at most L hops. Let Δ denote the maximum degree of the physical graph. In the worst case, the first hop has at most Δ choices,

while each subsequent hop has at most $\Delta - 1$ forward branches. Therefore, the number of explored states is upper bounded by $N_{\text{exp}} \leq 1 + \sum_{h=1}^L \Delta(\Delta - 1)^{h-1}$, which yields the worst-case time complexity $T_{\text{route}}(P_i) = \mathcal{O}(N_{\text{exp}}) = \mathcal{O}(\Delta(\Delta - 1)^{L-1})$. Because the hop count, cumulative delay, and score bound are updated incrementally during recursion, each state expansion only requires $\mathcal{O}(1)$ additional work under an adjacency-list implementation. For an SFC with $|E^{sfc}| = |V^{sfc}| - 1$ virtual links, the overall routing complexity is $\mathcal{O}(|E^{sfc}| \cdot \Delta(\Delta - 1)^{L-1})$. This bound is conservative. In practice, bandwidth pruning, delay pruning, and score-based branch-and-bound significantly reduce the number of explored states. Moreover, compared with explicitly materializing the full expansion tree, the DFS implementation only requires $\mathcal{O}(L)$ auxiliary space for the recursion stack and the current path, excluding the storage of the incumbent best solution.

2) *Placement policy*: Let $|\vec{f}|$ denote the input feature dimension and H the hidden width. With sparse graph operators, a single GCN layer on graph $G = (V, E)$ incurs $\mathcal{O}(|E| \cdot |\vec{f}| + |V| \cdot |\vec{f}| \cdot H)$ operations. Hence, two X -layer encoders for G^{pn} and G^{sfc} require $\mathcal{O}\left(X(|E^{pn}| \cdot |\vec{f}| + |V^{pn}| \cdot |\vec{f}| \cdot H)\right) + \mathcal{O}\left(X(|E^{sfc}| \cdot |\vec{f}| + |V^{sfc}| \cdot |\vec{f}| \cdot H)\right)$. The MLP components contribute $\mathcal{O}(|V^{pn}| \cdot H^2)$ operations, while feature construction costs $\mathcal{O}(|V^{pn}| \cdot |\vec{f}| + |V^{sfc}| \cdot |\vec{f}|)$. Therefore, the dominant cost of the placement module is determined by the graph encoders and the node-wise scoring network.

V. EXPERIMENTAL EVALUATION AND PERFORMANCE ANALYSIS

A. Baseline Methods

To comprehensively evaluate the performance of the proposed method, we compare it against the following baselines. **NodeRank** [21]: This heuristic algorithm computes a rank for each vertex based on its multi-dimensional resource attributes and greedily selects the vertex with the highest rank as the embedding target. **GCN-A2C**: In [32], Wang *et al.* employ Actor and Critic networks based on GCN with ReLU activation to address the multi-objective optimization of dynamic SFC deployment. **GCN-DDQN**: We further develop a variant based on work [13] that utilizes the Double Deep Q-Network (DDQN) framework [33] for training. DQN-style approaches, such as those in [34], have been widely adopted in VNE and orchestration tasks for SAGIN. **SFC-HPR (SPF)**: This variant keeps the same node deployment policy as **SFC-HPR**, while replacing the proposed L -HOP mechanism with a deterministic shortest-path-first (SPF) strategy for routing between consecutive VNFs.

B. Experimental Setup

Motivated by the dynamic behavior and cross-layer heterogeneity of multi-layer mega-constellation networks discussed in [35], we adopt a geometry-driven SAGIN topology to capture the network-layer dynamics that affect SFC placement and routing. The numerical settings follow prior studies [30], [16], and the corresponding parameter ranges are reported in Tab. II.

1) *SAGIN Network*: The physical substrate consists of satellite, aerial, and ground layers. Satellites are placed at a fixed altitude and follow a projected circular-orbit mobility model, with orbital phase $\phi_i(t) = \phi_i(0) + \omega t$, where $\omega = \sqrt{\mu_E/(R_E + h)^3}$. UAVs and vehicles follow Gauss-Markov mobility in the horizontal plane, whereas base stations remain stationary. After node positions are updated, all physical links are reconstructed according to link-type-specific geometric constraints. ISLs are determined by satellite distance, S2A and S2G links depend on coverage and elevation constraints, and aerial/ground links follow their communication ranges. Each link delay contains distance-dependent propagation delay and link-type-dependent jitter. The detailed parameter settings are listed in Tab. II. In addition, node and link resources are specified according to their classes within SAGIN [36]. Satellite and aerial nodes are assigned lower CPU and memory ranges than ground nodes, reflecting more limited onboard resource budgets. Link bandwidth ranges vary across space-related, aerial, and ground links. As a result, the topology differs from a homogeneous random graph in four respects: node mobility is explicitly modeled, links are reconstructed over time, cross-layer connectivity follows coverage and geometric constraints, and computing and communication resources are heterogeneous across layers.

2) *SFC Request*: To compare all methods under identical and controllable demand conditions, we generate each SFC request using the parameter ranges in [16]: the SFC length is uniformly sampled from $[2, 10]$, the TTL from $[200, 400]$, the CPU and memory demands of each VNF from $[10, 60]$, and the bandwidth demand of each virtual link from $[50, 150]$.

TABLE II
EXPERIMENTAL SETTINGS.

Topology Scales	
S20A20G40	20 satellites, 20 UAVs, 10 BSs, and 30 vehicles
S40A30G70	40 satellites, 30 UAVs, 20 BSs, and 50 vehicles
S60A40G100	60 satellites, 40 UAVs, 30 BSs, and 70 vehicles
Layered Settings	
Satellite projected-motion parameters	Satellite altitude: 550 km; service area: 500 km $\times$ 500 km; orbit radius: 6921 km; inclination: 53°; orbital planes: 5; angular velocity: $\omega = 1.0965 \times 10^{-3}$ rad/s; orbital period: 95.50 min; initial phase jitter: $\mathcal{U}[-2^\circ, 2^\circ]$
Velocity of UAV	$v_{\text{UAV}} \sim \mathcal{U}[0, 30]$ m/s; $\alpha = 0.85$
Velocity of vehicle	$v_{\text{veh}} \sim \mathcal{U}[0, 20]$ m/s; $\alpha = 0.75$
Max. distance of ISL	4000 km
Min. elevation angle	20°
Coverage ranges	S2A/S2G: 2500 km
Communication ranges	A2A: 80 km; A2G: 30 km; V2B: 8 km
Propagation delay	$t_p = \frac{d}{c}$, where d is the link distance and c is the speed of light
Distribution of jitter	S2S: $\mathcal{U}[10, 30]$ ms; S2A/S2G: $\mathcal{U}[15, 40]$ ms; A2A: $\mathcal{U}[1, 5]$ ms; A2G: $\mathcal{U}[1, 5]$ ms; G2G: $\mathcal{U}[0.25, 2.5]$ ms
Network Parameters	
Physical resources of S vertices	$\mathcal{U}[100, 500]$
Physical resources of A vertices	$\mathcal{U}[100, 500]$
Physical resources of G vertices	$\mathcal{U}[100, 1000]$
Processing delay of vertices	$\mathcal{U}[1, 20]$
Physical bandwidth of S2X links	$\mathcal{U}[500, 1000]$
Physical bandwidth of A2X links	$\mathcal{U}[500, 2000]$
Physical bandwidth of G2G links	$\mathcal{U}[500, 3000]$
SFC Parameters	
Length of SFC	$\mathcal{U}\{2, 3, \dots, 10\}$
Arrival rate of SFC requests	$\{0.1, 0.2, \dots, 1.0\}$
Time to live of SFC requests	$\mathcal{U}[200, 400]$
CPU demand of a virtual vertex	$\mathcal{U}[10, 60]$
Memory demand of a virtual vertex	$\mathcal{U}[10, 60]$
Bandwidth demand of a virtual link	$\mathcal{U}[50, 150]$

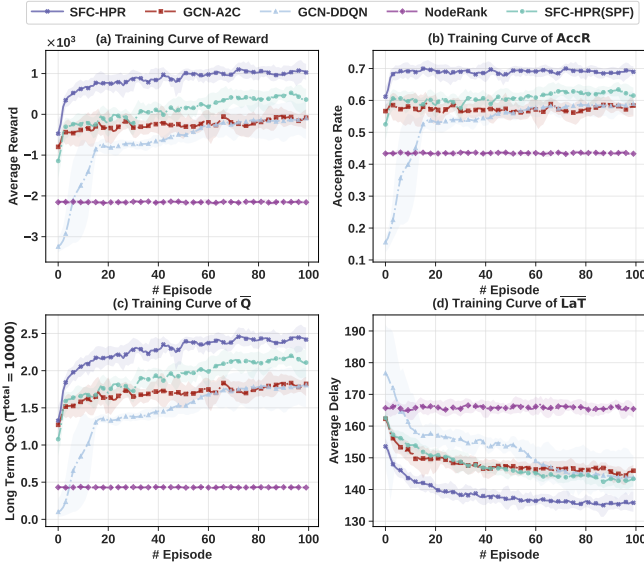


Fig. 3. Training performance on the S40A30G70 topology. (a) Average reward. (b) Acceptance rate. (c) Long-term Resource-Latency QoS over $T^{\text{total}} = 10000$. (d) Average latency of accepted SFCs.

C. Training Performance

All learning based methods are independently trained on each topology scale. Fig. 3 reports training results on the medium-scale S40A30G70 topology. SFC-HPR provides the best overall balance among compared methods. After training, Fig. 3 (b)-(d) report $AccR = 0.688 \pm 0.010$, $\bar{Q} = 2.395 \pm$

0.082 , and $\bar{LaT} = 136.85 \pm 3.02$, respectively. Relative to SFC-HPR (SPF), SFC-HPR increases $AccR$ from 0.623 to 0.688 in Fig. 3 (b) and $\bar{Q}$ from 2.127 to 2.395 in Fig. 3 (c), while reducing $\bar{LaT}$ from 144.14 to 136.85 in Fig. 3 (d). Because both methods share the same placement policy, the performance gap reflects the effect of the bounded-hop, QoS-aware routing stage rather than VNF-placement differences.

GCN-A2C and GCN-DDQN reach lower final acceptance rates and QoS values in Fig. 3 (b) and Fig. 3 (c). Both methods end with $AccR$ values of approximately 0.59 and $\bar{Q}$ values below 1.88. SFC-HPR therefore improves both embedding feasibility and accumulated service utility. NodeRank performs worst overall, with $AccR = 0.436$, $\bar{Q} = 0.433$, and $\bar{LaT} = 165.98$ in Fig. 3 (b)-(d).

D. Reward Ablation

Tab. III compares the full reward in Eq. (21) with two variants, only acceptance rate (OnlyAcc) and without bandwidth (w/o BW). The full reward attains the highest $\bar{Q}$, reaching 2.614 ± 0.098 , while maintaining $\bar{LaT} = 133.04 \pm 1.01$. OnlyAcc reaches a higher $AccR$ than the full reward, at 0.741 ± 0.012 and 0.719 ± 0.022 , respectively. However, its $\bar{Q}$ falls to 2.193 ± 0.066 , and its $\bar{LaT}$ rises to 143.32 ± 2.60 . Maximizing acceptance alone therefore does not maximize

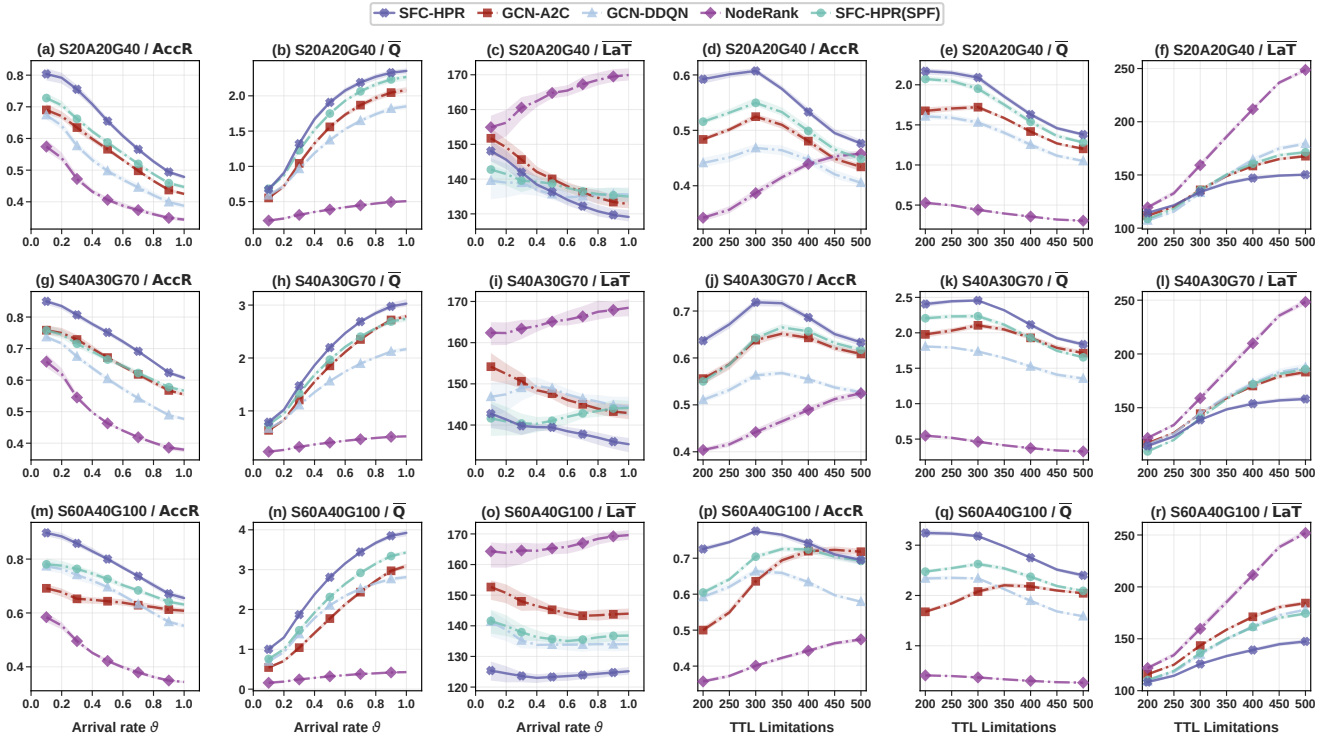


Fig. 4. Testing performance on dynamic SAGIN topologies of different scales. The left nine panels evaluate different request arrival rates ϑ : (a)-(c) for S20A20G40, (g)-(i) for S40A30G70, and (m)-(o) for S60A40G100. Each group reports $AccR$, $\bar{Q}$, and $\bar{LaT}$. The right nine panels evaluate different TTL limits: (d)-(f) for S20A20G40, (j)-(l) for S40A30G70, and (p)-(r) for S60A40G100, with the metrics reported in the same order.

Resource-Latency utility. The w/o BW variant produces the lowest $\bar{LaT}$, 132.45 ± 2.54 , but its $AccR$ and $\bar{Q}$ decrease to 0.682 ± 0.031 and 2.319 ± 0.147 , respectively. Removing the bandwidth term keeps the average resource-latency provisioning close to the full reward, 7.255 ± 0.186 versus 7.213 ± 0.080 , but reduces average bandwidth efficiency from 0.841 ± 0.015 to 0.779 ± 0.027 . The reward must therefore balance acceptance, latency, and resource use rather than optimize acceptance alone. The full reward follows the objective in Eq. (16) by jointly accounting for $AccR$, $-\bar{LaT}$, and $\bar{Q}$ under strict TTL constraints.

TABLE III
REWARD ABLATION ON S40A30G70.

Metric	Full	w/o BW	OnlyAcc
$AccR$	0.719 ± 0.022	0.682 ± 0.031	0.741 ± 0.012
$\bar{LaT}$	133.04 ± 1.01	132.45 ± 2.54	143.32 ± 2.60
Avg. $\frac{\psi^{sfc}}{T^{sfc}}$	7.213 ± 0.080	7.255 ± 0.186	6.160 ± 0.144
Avg. η_{bw}^{sfc}	0.841 ± 0.015	0.779 ± 0.027	0.802 ± 0.018
$\bar{Q}$	2.614 ± 0.098	2.319 ± 0.147	2.193 ± 0.066

E. Testing Under Different Dynamic SAGIN Topologies

1) *Testing Under Different Arrival Rates*: Fig. 4 (a)-(c), (g)-(i), and (m)-(o) report testing results across topology scales and arrival rates. On S20A20G40, Fig. 4 (a) and Fig. 4 (b) show that SFC-HPR achieves average $AccR = 0.639$ and average $\bar{Q} = 1.763$ across all arrival rates. These values improve $AccR$ by 10.57% and $\bar{Q}$ by 5.96% over SFC-HPR (SPF). In Fig. 4 (c), SFC-HPR and GCN-DDQN have nearly identical latency on the smallest topology, 136.64 and 136.62,

respectively. The compact topology may provide too few routing alternatives for substantial latency differences. The gap between SFC-HPR and the baselines becomes clearer as the topology grows. On S40A30G70, Fig. 4 (g)-(i) show that SFC-HPR achieves $AccR = 0.732$ and $\bar{Q} = 2.127$, compared with 0.659 and 1.922, respectively, for SFC-HPR (SPF). SFC-HPR also reaches $\bar{LaT} = 138.81$, lower than those for SFC-HPR (SPF), GCN-A2C, GCN-DDQN, and NodeRank. On S60A40G100, Fig. 4 (m)-(o) show that SFC-HPR achieves $AccR = 0.780$ and $\bar{Q} = 2.732$, while reducing $\bar{LaT}$ to 124.18. Relative to SFC-HPR (SPF), its $AccR$ gains on the three topologies are 10.57%, 11.15%, and 9.68%, respectively. The arrival-rate curves in Fig. 4 (a), (g), and (m) show increasing resource pressure. As ϑ rises, $AccR$ decreases because more SFC requests compete for limited vertex and link resources. By contrast, Fig. 4 (b), (h), and (n) show that $\bar{Q}$ increases because the cumulative utility of accepted services grows over T^{total} . The two metrics therefore capture different aspects of performance: $AccR$ measures request-level feasibility, whereas $\bar{Q}$ measures long-term service utility.

2) *Testing Under Different TTL Limits*: Fig. 4 (d)-(f), (j)-(l), and (p)-(r) report results under different TTL limits. Averaged over three topology scales, seven TTL limits, and five random seeds, SFC-HPR achieves the highest overall acceptance rate and QoS: $AccR = 0.6518$ and $\bar{Q} = 2.3005$. The corresponding $AccR/\bar{Q}$ values are 0.6013/2.0367 for SFC-HPR (SPF), 0.5794/1.8132 for GCN-A2C, 0.5326/1.6668 for GCN-DDQN, and 0.4300/0.3898 for NodeRank. Relative to SFC-HPR (SPF), the strongest competitor, SFC-HPR improves $AccR$ by 0.0505, or 8.40%, and $\bar{Q}$ by 0.2638, or 12.95%. The plots show this advantage is not confined to

TABLE IV
RUNTIME OVERHEAD OF SINGLE EPISODE.

Method	S20A20G40	S40A30G70	S60A40G100
NodeRank	79.460 ± 6.902	133.664 ± 10.361	175.759 ± 12.034
GCN-DDQN	223.582 ± 27.288	244.434 ± 23.173	272.824 ± 19.079
GCN-A2C	268.617 ± 5.600	280.440 ± 17.571	306.307 ± 28.348
SFC-HPR (SPF)	328.181 ± 12.091	349.255 ± 15.581	380.221 ± 18.951
SFC-HPR	389.743 ± 25.713	405.989 ± 32.853	429.529 ± 29.834

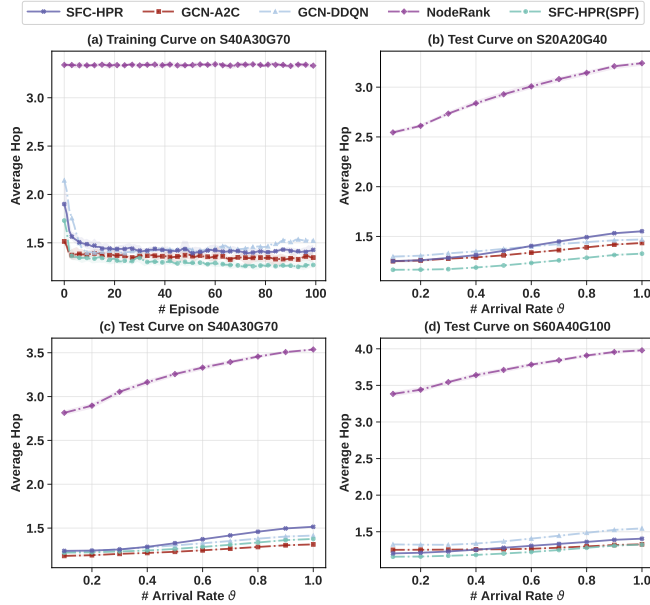


Fig. 5. Short-hop characteristics of successful embeddings.

a single configuration. Increasing TTL^{sfc} from 200 to 500 reduces latency-violation failures but does not monotonically improve QoS. For SFC-HPR, average latency-exceedance failures fall from 1894.67 to 61.67. Over the same range, Fig. 4 (d), (j), and (p) show that average $AccR$ increases from 0.6051 at $TTL^{sfc} = 200$ to a peak of 0.7145 at $TTL^{sfc} = 300$, then decreases to 0.5842 at $TTL^{sfc} = 500$. Fig. 4 (e), (k), and (q) show a similar pattern for $\bar{Q}$, which peaks at 2.7323 at $TTL^{sfc} = 250$ and falls to 1.7847 at $TTL^{sfc} = 500$. Relaxing the latency constraint removes one source of infeasibility but makes bandwidth contention the main bottleneck. A larger TTL^{sfc} therefore does not necessarily make the embedding problem easier overall. SFC-HPR remains competitive across all three topology scales, and its margin generally widens as the network grows. Relative to SFC-HPR (SPF), average gains in $\bar{Q}$ are 6.02% on S20A20G40, 9.85% on S40A30G70, and 20.51% on S60A40G100, as shown in Fig. 4 (e), (k), and (q). The corresponding average latency reductions are 4.15%, 5.39%, and 9.54% in Fig. 4 (f), (l), and (r). Learned placement with quality-aware routing is more useful when the substrate offers enough spatial diversity for alternative routing decisions. The only clear acceptance-rate reversals occur in Fig. 4 (p) on the largest topology at $TTL^{sfc} = 450$ and 500. At these points, GCN-A2C achieves 0.7237 and 0.7126, while SFC-HPR achieves 0.7126 and 0.6779, respectively. SFC-HPR performs best on average, but not at every operating point under relaxed TTL limits.

3) *Runtime overhead*: Tab. IV further reports runtime overhead (seconds) on single episode. NodeRank is fastest on every topology, averaging 79.460, 133.664, and 175.759 s on S20A20G40, S40A30G70, and S60A40G100, respectively. GCN-DDQN is consistently lighter than GCN-A2C, reducing runtime from 268.617 to 223.582 s on S20A20G40, from 280.440 to 244.434 s on S40A30G70, and from 306.307 to 272.824 s on S60A40G100. SFC-HPR incurs the largest runtime overhead among compared methods, increasing from 389.743 s on S20A20G40 to 429.529 s on S60A40G100.

F. Analysis of short hop

Across all arrival rates, Fig. 5 (b)-(d) show that SFC-HPR's average Path Hop remains within a narrow range: 1.390 on S20A20G40, 1.362 on S40A30G70, and 1.298 on S60A40G100. These values are well below NodeRank's Path Hop values of 2.933, 3.238, and 3.718 on the same topologies. SFC-HPR does not always minimize Path Hop. Fig. 5 (a)-(d) show that SFC-HPR (SPF) sometimes selects slightly shorter paths because it directly chooses the shortest available route. Instead, SFC-HPR uses the bounded-hop neighborhood to limit the candidate set, then selects a path using the QoS-aware routing score in Eq. (23). The results support using a small hop bound to control the search space.

G. Sensitivity analysis of L

We evaluate the sensitivity to L on S40A30G70 by varying L from 1 to 7. As shown in Tab. V, increasing L from 1 to 3 raises $AccR$ from 0.660 to 0.726 and $\bar{Q}$ from 2.163 to 2.510, while reducing $\bar{LaT}$ from 146.86 to 138.32. The $L = 1$ setting is too restrictive because it removes useful feasible paths from the search space.

Performance largely saturates beyond $L = 3$. As L increases from 3 to 7, $AccR$ changes only from 0.726 to 0.727, while $\bar{Q}$ decreases slightly from 2.510 to 2.487. Over the same range, routing overhead rises from 0.941 ms to 18.709 ms. A larger hop bound therefore expands the search space and increases computation without producing a comparable gain in embedding quality. Compared with SFC-HPR (SPF), $L = 3$ improves both $AccR$ and $\bar{Q}$ while keeping routing overhead below that of the SPF baseline. Among the tested values, $L = 3$ gives the best balance between embedding performance and routing cost.

TABLE V
SENSITIVITY ANALYSIS OF L .

Setting	$AccR$	$\bar{Q}$	$\bar{LaT}$	Overhead (ms)
$L = 1$	0.660 ± 0.006	2.163 ± 0.015	146.86 ± 1.07	1.312 ± 0.036
$L = 2$	0.715 ± 0.009	2.436 ± 0.018	141.69 ± 1.20	0.435 ± 0.014
$L = 3$	0.726 ± 0.004	2.510 ± 0.014	138.32 ± 1.21	0.941 ± 0.009
$L = 4$	0.725 ± 0.005	2.487 ± 0.028	138.98 ± 1.21	2.509 ± 0.079
$L = 5$	0.725 ± 0.005	2.491 ± 0.018	138.49 ± 0.62	5.582 ± 0.113
$L = 6$	0.724 ± 0.008	2.498 ± 0.020	138.26 ± 0.90	10.545 ± 0.173
$L = 7$	0.727 ± 0.009	2.487 ± 0.023	138.38 ± 1.07	18.709 ± 0.422
SPF	0.643 ± 0.008	2.225 ± 0.037	141.54 ± 0.76	5.235 ± 0.052

VI. CONCLUSION

This paper addresses the problem of latency-constrained SFC Placement and Routing in SAGIN. We propose SFC-HPR,

a two-stage hierarchical framework designed to maximize the SFC acceptance ratio and resource-based QoS. SFC-HPR employs a DRL-based placement agent that utilizes parallel GCN encoders to extract features from both physical and virtual networks. These features are combined with SFC placement-order information to construct an enhanced state representation. The agent generates placement decisions based on these representations and is trained using the A2C algorithm. Furthermore, motivated by the short hop characteristic of SFC placement, we design a hop-count routing scheme that aligns with the QoS objective. We also incorporate a safeguard mechanism to ensure routing performance, thereby preserving embedding quality without significantly increasing computational complexity. Experimental results demonstrate that SFC-HPR outperforms mainstream baselines in SAGIN scenarios. Our findings reveal that the joint impact of an effective placement strategy and an appropriate routing mechanism on SFC embedding performance is both pronounced and complex. Consequently, the interplay between service placement and routing warrants further investigation.

REFERENCES

- [1] J. Liu, Y. Shi, Z. M. Fadlullah, and N. Kato, "Space-air-ground integrated network: A survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2714–2741, 2018.
- [2] Y. Zhang, X. Wang, Y. Gang, J. Wang, S. Wu, P. Zhang, and Y. Shi, "6g sagin information transmission model," *IEEE Commun. Mag.*, vol. 63, no. 6, pp. 98–105, 2025.
- [3] H. Cui, J. Zhang, Y. Geng, Z. Xiao, T. Sun, N. Zhang, J. Liu, Q. Wu, and X. Cao, "Space-air-ground integrated network (sagin) for 6g: Requirements, architecture and challenges," *China Commun.*, vol. 19, no. 2, pp. 90–108, 2022.
- [4] P. Zhang, G. Wang, S. Chen, Y. Yu, and L. Chen, "The application of mobile edge computing in the space-air-ground integrated network," *Journal of Intelligent Computing and Networking*, vol. 1, no. 1, p. 71–97, Aug. 2025.
- [5] S. Tao, M. Yuan, Q. Wu, R. Wang, and J. Hao, "Generative ai-aided vertical handover decision in sagin for iot with integrated sensing and communication," *IEEE Internet Things J.*, vol. 12, no. 10, pp. 13 297–13 310, 2025.
- [6] H. Jia, Y. Wang, and W. Wu, "Dynamic resource allocation for remote iot data collection in sagin," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20 575–20 589, 2024.
- [7] D. Bhamare, R. Jain, M. Samaka, and A. Erbad, "A survey on service function chaining," *Journal of Network and Computer Applications*, vol. 75, pp. 138–155, 2016.
- [8] P. Zhang, Y. Zhang, N. Kumar, M. Guizani, A. Barnawi, and W. Zhang, "Energy-aware positioning service provisioning for cloud-edge-vehicle collaborative network based on drl and service function chain," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3424–3435, 2024.
- [9] Q. Wang, X. Chen, X. Jin, X. Li, D. Chen, and X. Qin, "Enhancing trustworthiness of internet of vehicles in space-air-ground-integrated networks: Attestation approach," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5992–6002, 2022.
- [10] H. Liang, Z. Yang, G. Zhang, and H. Hou, "Resource allocation for space-air-ground integrated networks: A comprehensive review," *Journal of Communications and Information Networks*, vol. 9, no. 1, pp. 1–23, 2024.
- [11] J. Li, W. Shi, H. Wu, S. Zhang, and X. Shen, "Cost-aware dynamic sfc mapping and scheduling in sdn/nfv-enabled space-air-ground-integrated networks for internet of vehicles," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5824–5838, 2022.
- [12] Y. Zhu, Y. Guo, C. Chen, Y. Li, and P. He, "A hierarchical sdn controller synergy approach for dynamic load balancing in sagin networks: Enhanced clustering and optimization via dpc-k-means and isao algorithms," *IEEE Trans. Aerosp. Electron. Syst.*, pp. 1–19, 2025.
- [13] Z. Jia, Y. Cao, L. He, Q. Wu, Q. Zhu, D. Niyato, and Z. Han, "Service function chain dynamic scheduling in space-air-ground integrated networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 7, pp. 11 235–11 248, 2025.
- [14] P. Zhang, Z. Li, M. Guizani, N. Kumar, K. Yu, and J. Wang, "Energy aware space-air-ground integrated network resource orchestration algorithm," *IEEE Trans. Veh. Technol.*, vol. 73, no. 12, pp. 18 950–18 960, 2024.
- [15] Y. Liu, P. Wang, and S. Wang, "Robustness plugin for military target recognition algorithms in on-orbit satellite environments," *Journal of Intelligent Computing and Networking*, vol. 1, no. 1, p. 54–62, Aug. 2025.
- [16] P. Zhang, S. Chen, J. Fan, L. Tan, and C. Jiang, "Adaptive orchestration of service function chains in sagin-mec via graph reinforcement learning," *IEEE Trans. Mobile Comput.*, pp. 1–13, 2026.
- [17] G. Wang, S. Zhou, S. Zhang, Z. Niu, and X. Shen, "Sfc-based service provisioning for reconfigurable space-air-ground integrated networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 7, pp. 1478–1489, 2020.
- [18] I. Jang, D. Suh, S. Pack, and G. Dán, "Joint optimization of service function placement and flow distribution for service function chaining," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2532–2541, 2017.
- [19] J. Pei, P. Hong, K. Xue, and D. Li, "Resource aware routing for service function chains in sdn and nfv-enabled network," *IEEE Trans. Serv. Comput.*, vol. 14, no. 4, pp. 985–997, 2021.
- [20] Y. Duan, Y. Lu, S. Shen, S. Yu, P. Zhang, W. Zhang, and K. K. Iqbal, "Nflcs: An service function chain path optimization strategy based on network-functional layout clustering," *IEEE Trans. Veh. Technol.*, vol. 72, no. 8, pp. 10 813–10 825, 2023.
- [21] P. Zhang, H. Yao, and Y. Liu, "Virtual network embedding based on computing, network, and storage resource constraints," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, 2018.
- [22] P. Zhang, C. Wang, C. Jiang, and A. Benslimane, "Security-aware virtual network embedding algorithm based on reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1095–1105, 2021.
- [23] V. Eramo, E. Miucci, M. Ammar, and F. G. Lavacca, "An approach for service function chain routing and virtual function network instance migration in network function virtualization architectures," *IEEE/ACM Trans. Netw.*, vol. 25, no. 4, pp. 2008–2025, 2017.
- [24] S. Le, Y. Wu, Y. Guo, and C. D. Vecchio, "Game theoretic approach for a service function chain routing in nfv with coupled constraints," *IEEE Trans. Circuits Syst. II*, vol. 68, no. 12, pp. 3557–3561, 2021.
- [25] M. Ghaznavi, N. Shahriar, S. Kamali, R. Ahmed, and R. Boutaba, "Distributed service function chaining," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2479–2489, 2017.
- [26] W. Chen, X. Yin, Z. Wang, X. Shi, and J. Yao, "Placement and routing optimization problem for service function chain: State of art and future opportunities," in *International Conference on Artificial Intelligence and Security*. Springer, 2020, pp. 176–188.
- [27] Y. Wang, C.-K. Huang, S.-H. Shen, and G.-M. Chiu, "Adaptive placement and routing for service function chains with service deadlines," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 3021–3036, 2021.
- [28] X. Liu, S. Zhao, X. Wang, and C. Zhou, "Lpule: An efficient algorithm for service function chain placement and routing with delay guarantee," *Computer Networks*, vol. 253, p. 110728, 2024.
- [29] Y. Gong, H. Yao, Z. Xiong, D. Yu, X. Cheng, C. Yuen, M. Bennis, and M. Debbah, "Multi-modal federated learning based resources convergence for satellite-ground twin networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 4104–4117, 2025.
- [30] T. Wang, Q. Fan, C. Wang, L. Yang, L. Ding, N. J. Yuan, and H. Xiong, "Flagvne: A flexible and generalizable reinforcement learning framework for network resource allocation," *arXiv preprint arXiv:2404.12633*, 2024.
- [31] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, 2021.
- [32] S. Wang, H. Cao, L. Yang, S. Garg, G. Kaddoum, and M. Alrashoud, "Gcn-based multi-agent deep reinforcement learning for dynamic service function chain deployment in iot," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 6105–6118, 2024.
- [33] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.
- [34] I. Maity, T. X. Vu, and S. Chatzinotas, "Revine: Reinforcement learning-based virtual network embedding in satellite-terrestrial networks," *IEEE Trans. Commun.*, vol. 72, no. 10, pp. 6316–6327, 2024.
- [35] N. Wu, M. Wang, T. Zhang, Y. Chang, and H. Yin, "Hierarchical intelligent routing under multi-layer mega-constellation network management architecture," *IEEE Wireless Commun.*, vol. 33, no. 3, pp. 39–46, 2026.
- [36] Y. Gong, D. Yu, H. Yao, X. Cheng, A. Nallanathan, and G. K. Karagiannis, "Multi-modal learning-based multi-task offloading schemes for satellite-ground integrated networks," *IEEE Trans. Wireless Commun.*, vol. 24, no. 7, pp. 5635–5647, 2025.